

Prof. Ashwani Mishra
Assistant Professor
CSE Department

Unit 6: Web Application Integration Techniques

1. Introduction to AsyncTask

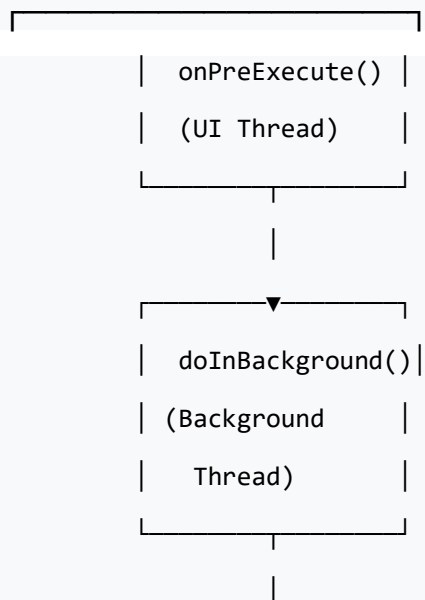
What is AsyncTask?

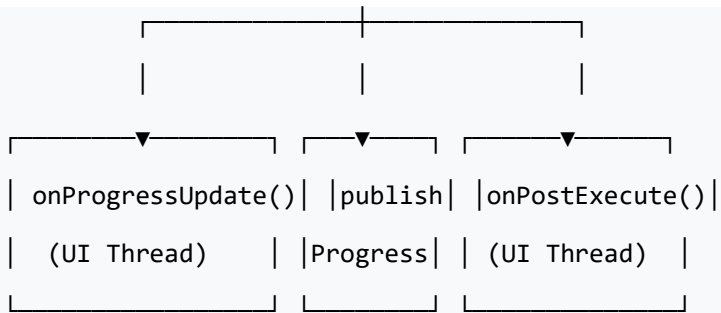
AsyncTask is an Android class that helps perform background operations and publish results on the UI thread without having to manipulate threads and handlers manually.

Purpose of AsyncTask

- Perform long-running operations in background
- Keep UI thread responsive
- Update UI from background thread
- Handle threading efficiently

AsyncTask Lifecycle Diagram





AsyncTask Structure

```
private class MyAsyncTask extends AsyncTask<Params, Progress, Result> {
```

```
    // Parameters: Type of parameters sent to task
    // Progress: Type of progress units published
    // Result: Type of result after background computation
```

```
@Override
```

```
protected void onPreExecute() {
```

```
    // Runs on UI thread before background task
```

```
    // Used for setup (show progress bar)
```

```
    super.onPreExecute();
```

```
}
```

```
@Override
```

```
protected Result doInBackground(Params... params) {
```

```
    // Runs on background thread
```

```
    // Perform long operations here
```

```
    // Call publishProgress() to update UI
```

```
    return result;
```

```
}
```

```
@Override
```

```
protected void onProgressUpdate(Progress... values) {
```

```

        // Runs on UI thread

        // Update UI with progress
        super.onProgressUpdate(values);
    }

    @Override
    protected void onPostExecute(Result result) {
        // Runs on UI thread after completion
        // Update UI with result
        super.onPostExecute(result);
    }
}

```

Example: Downloading Data with AsyncTask

```

public class DownloadTask extends AsyncTask<String, Integer, String> {

    @Override
    protected void onPreExecute() {
        progressBar.setVisibility(View.VISIBLE);
    }

    @Override
    protected String doInBackground(String... urls) {
        String result = "";
        try {
            URL url = new URL(urls[0]);

            HttpURLConnection connection = (HttpURLConnection)
url.openConnection();

            InputStream in = connection.getInputStream();

            BufferedReader reader = new BufferedReader(new
InputStreamReader(in));

```

```

        String line;

        while ((line = reader.readLine()) != null) {
            result += line;

            publishProgress(50); // Update progress
        }
    } catch (Exception e) {
        e.printStackTrace();
    }

    return result;
}

@Override
protected void onProgressUpdate(Integer... values) {
    progressBar.setProgress(values[0]);
}

@Override
protected void onPostExecute(String result) {
    textView.setText(result);
    progressBar.setVisibility(View.GONE);
}
}

// Execute the task
new DownloadTask().execute("https://api.example.com/data");

```

2. Communication with Web API

What is Web API?

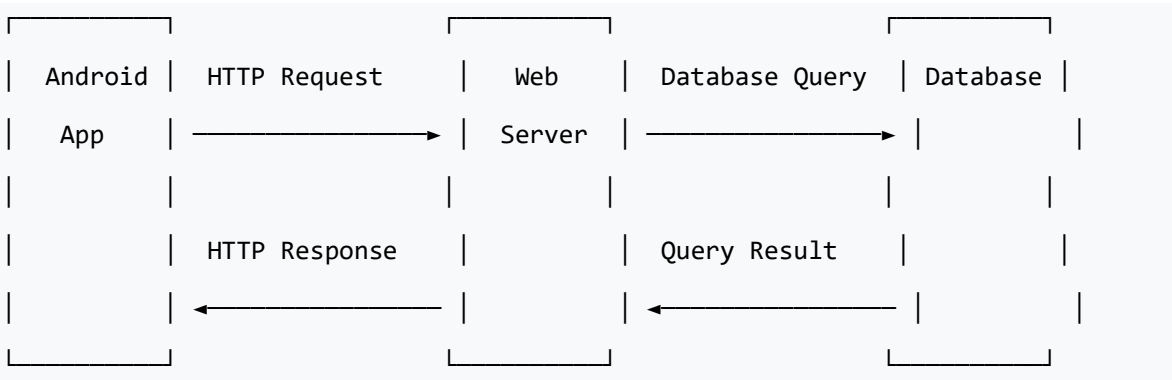
Web API (Application Programming Interface) is an interface that allows applications to communicate with each other over the internet using HTTP protocols.

HTTP Methods

Method	Description	Use Case
GET	Retrieve data	Fetching user profiles
POST	Create new resource	Creating new user
PUT	Update entire resource	Updating user details
PATCH	Partial update	Updating user email
DELETE	Remove resource	Deleting user account

API Communication Flow Diagram

text



Making HTTP Requests (Traditional Way)public class ApiClient {

```
public static String makeGetRequest(String urlString) {
    StringBuilder result = new StringBuilder();

    try {
        URL url = new URL(urlString);
        HttpURLConnection connection = (HttpURLConnection)
url.openConnection();

        connection.setRequestMethod("GET");
        connection.setConnectTimeout(5000);
        connection.setReadTimeout(5000);

        int responseCode = connection.getResponseCode();
```

```

        if (responseCode == HttpURLConnection.HTTP_OK) {
            BufferedReader reader = new BufferedReader(
                new InputStreamReader(connection.getInputStream()));
            String line;
            while ((line = reader.readLine()) != null) {
                result.append(line);
            }
            reader.close();
        }

        connection.disconnect();

    } catch (Exception e) {
        e.printStackTrace();
    }

    return result.toString();
}

public static String makePostRequest(String urlString, String jsonData) {
    // Similar implementation with POST method
    try {
        URL url = new URL(urlString);
        HttpURLConnection connection = (HttpURLConnection)
url.openConnection();

        connection.setRequestMethod("POST");
        connection.setRequestProperty("Content-Type", "application/json");
        connection.setDoOutput(true);
    }

```

```

        // Write JSON data to output stream
        OutputStream os = connection.getOutputStream();
        os.write(jsonData.getBytes());
        os.flush();

        // Read response
        // ... similar to GET method

    } catch (Exception e) {
        e.printStackTrace();
    }
    return "";
}
}

```

Using AsyncTask with Web API

```

public class FetchDataTask extends AsyncTask<String, Void, String> {

    private ApiCallback callback;

    public FetchDataTask(ApiCallback callback) {
        this.callback = callback;
    }

    @Override
    protected String doInBackground(String... params) {
        return ApiClient.makeGetRequest(params[0]);
    }

    @Override
    protected void onPostExecute(String result) {

```

```
        callback.onSuccess(result);
    }

    interface ApiCallback {
        void onSuccess(String data);
        void onError(String error);
    }
}
```

3. Introduction to JSON Data

What is JSON?

JSON (JavaScript Object Notation) is a lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate.

JSON Structure

text

JSON Structure:

- ├─ Object: { }
- | └─ Key: "name"
- | └─ Value: "John"
- ├─ Array: []
- | └─ Element 1
- | └─ Element 2
- └─ Data Types:
 - ├─ String: "text"
 - ├─ Number: 123, 45.6
 - ├─ Boolean: true/false
 - ├─ Array: [value1, value2]
 - ├─ Object: {key: value}
 - └─ Null: null

JSON Syntax Examples


```
// Simple JSON Object
```

```
{  
  "name": "John Doe",  
  "age": 30,  
  "isStudent": false,  
  "height": 5.9  
}
```

```
// JSON with Array
```

```
{  
  "users": [  
    {  
      "id": 1,  
      "name": "Alice",  
      "email": "alice@example.com"  
    },  
    {  
      "id": 2,  
      "name": "Bob",  
      "email": "bob@example.com"  
    }  
  ]  
}
```

```
// Nested JSON
```

```
{  
  "company": "Tech Corp",  
  "location": {  
    "city": "New York",  
    "country": "USA",  
  }  
}
```

```
        "coordinates": {
            "lat": 40.7128,
            "lng": -74.0060
        },
        "employees": [
            {
                "name": "John",
                "department": "Engineering",
                "skills": ["Java", "Kotlin", "Python"]
            }
        ]
    }
```

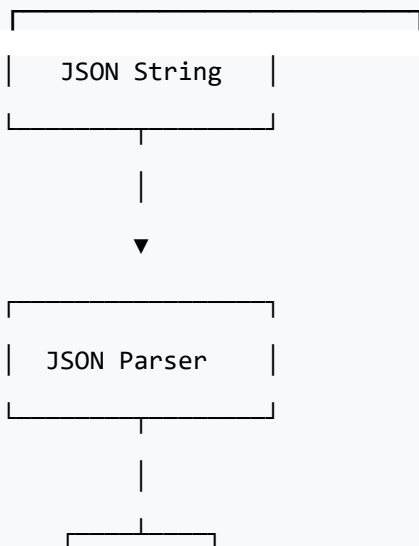
4. JSON Parsing

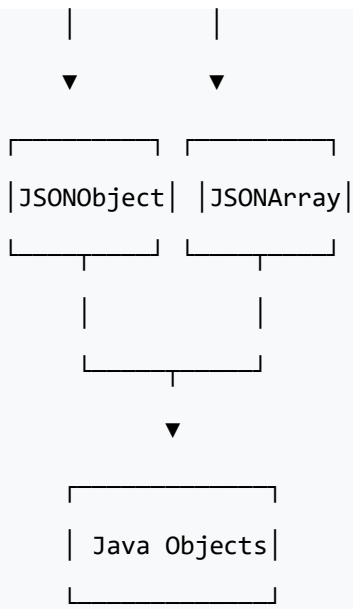
JSON Parsing in Android

Android provides two main ways to parse JSON:

1. **JSONObject and JSONArray** (Built-in)
2. **Gson Library** (Google's library)

JSON Parsing Flow Diagram





Method 1: Using JSONObject (Built-in)

```
public class JSONParser {

    public static User parseUser(String jsonString) {
        User user = new User();

        try {
            JSONObject jsonObject = new JSONObject(jsonString);

            // Extract primitive values
            user.setId(jsonObject.getInt("id"));
            user.setName(jsonObject.getString("name"));
            user.setEmail(jsonObject.getString("email"));
            user.setAge(jsonObject.getInt("age"));

            // Extract nested object
            JSONObject addressObj = jsonObject.getJSONObject("address");
            Address address = new Address();
```

```

        address.setStreet(addressObj.getString("street"));
        address.setCity(addressObj.getString("city"));
        address.setZipCode(addressObj.getString("zipcode"));
        user.setAddress(address);

        // Extract array
        JSONArray hobbiesArray = jsonObject.getJSONArray("hobbies");
        List<String> hobbies = new ArrayList<>();
        for (int i = 0; i < hobbiesArray.length(); i++) {
            hobbies.add(hobbiesArray.getString(i));
        }
        user.setHobbies(hobbies);

    } catch (JSONException e) {
        e.printStackTrace();
    }

    return user;
}

public static List<User> parseUserList(String jsonString) {
    List<User> userList = new ArrayList<>();

    try {
        JSONArray jsonArray = new JSONArray(jsonString);

        for (int i = 0; i < jsonArray.length(); i++) {
            JSONObject userObj = jsonArray.getJSONObject(i);
            User user = new User();
            user.setId(userObj.getInt("id"));

```

```

        user.setName(userObj.getString("name"));
        userList.add(user);
    }

    } catch (JSONException e) {
        e.printStackTrace();
    }

    return userList;
}
}

```

Method 2: Using Gson Library

Add dependency in build.gradle:

gradle

```

dependencies {
    implementation 'com.google.code.gson:gson:2.10.1'
}

```

Model Classes:

public class User {

```

    private int id;
    private String name;
    private String email;
    private Address address;
    private List<String> hobbies;

```

// Getters and setters

}

public class Address {

```

    private String street;

```

```
private String city;

private String zipcode;


// Getters and setters
}
```

Parsing with Gson:

```
public class GsonParser {

    private Gson gson = new Gson();


    // Parse single object
    public User parseUser(String jsonString) {
        return gson.fromJson(jsonString, User.class);
    }


    // Parse list of objects
    public List<User> parseUserList(String jsonString) {
        Type userListType = new TypeToken<ArrayList<User>>(){}.getType();
        return gson.fromJson(jsonString, userListType);
    }

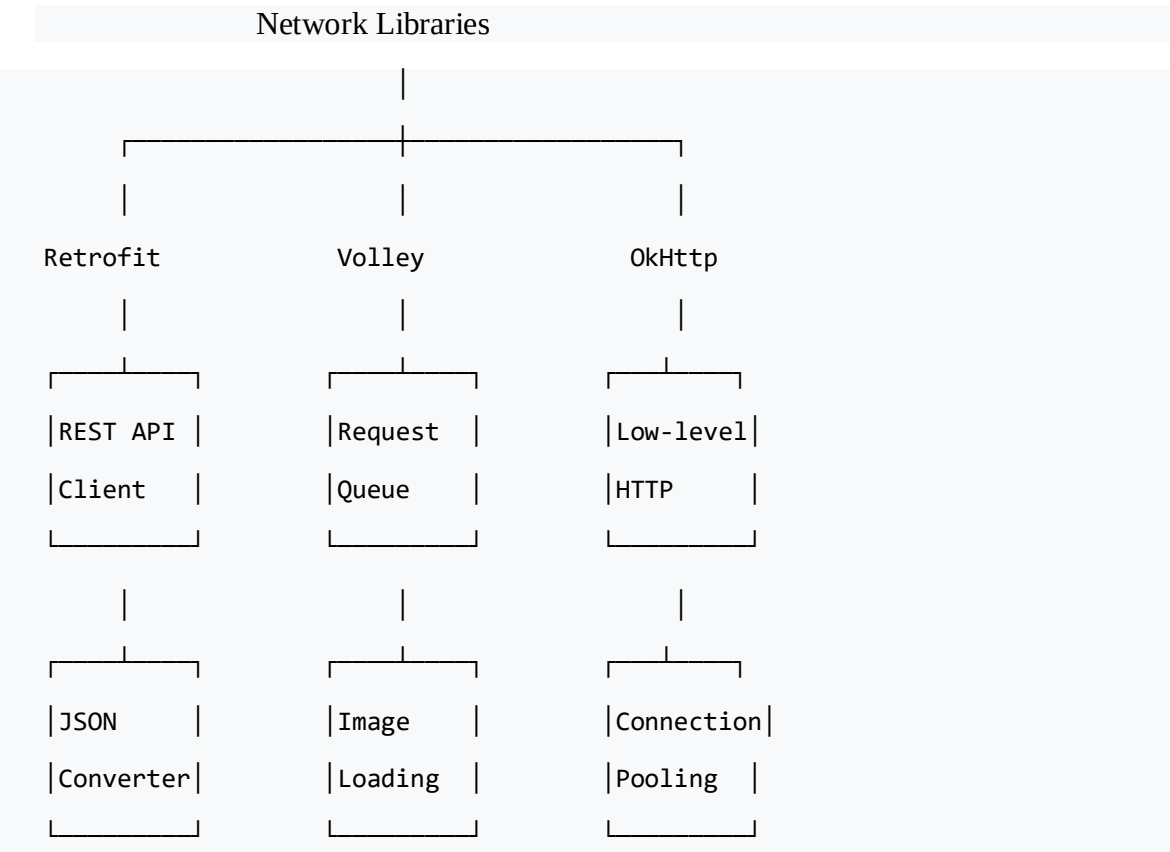

    // Convert object to JSON
    public String toJson(User user) {
        return gson.toJson(user);
    }
}
```

5. Third-Party Libraries to Fetch Network Data

Popular Libraries

Library	Developer	Features
Retrofit	Square	Type-safe HTTP client, REST API support
Volley	Android Team	Request queuing, caching, image loading
OkHttp	Square	HTTP/2 support, connection pooling
Picasso	Square	Image loading and caching
Glide	Bumptech	Image loading, GIF support

Library Comparison Diagram



1. Retrofit Implementation

Add dependencies:

gradle

```
dependencies {
    implementation 'com.squareup.retrofit2:retrofit:2.9.0'
    implementation 'com.squareup.retrofit2:converter-gson:2.9.0'
    implementation 'com.squareup.okhttp3:logging-interceptor:4.11.0'
}
```

Create API Interface:

```
public interface ApiService {
```

```
    @GET("users")
    Call<List<User>> getUsers();

    @GET("users/{id}")
    Call<User> getUser(@Path("id") int userId);

    @POST("users")
    Call<User> createUser(@Body User user);

    @PUT("users/{id}")
    Call<User> updateUser(@Path("id") int userId, @Body User user);

    @DELETE("users/{id}")
    Call<Void> deleteUser(@Path("id") int userId);

    @GET("posts")
    Call<List<Post>> getPosts(@Query("userId") int userId);

    @GET("search")
    Call<SearchResult> search(@Query("q") String query);
}
```

Create Retrofit Client:

```
public class RetrofitClient {

    private static final String BASE_URL =
    "https://jsonplaceholder.typicode.com/";

    private static Retrofit retrofit = null;
```



```

public static ApiService getApiService() {
    if (retrofit == null) {
        // Add logging interceptor for debugging
        HttpLoggingInterceptor logging = new HttpLoggingInterceptor();
        logging.setLevel(HttpLoggingInterceptor.Level.BODY);

        OkHttpClient client = new OkHttpClient.Builder()
            .addInterceptor(logging)
            .connectTimeout(30, TimeUnit.SECONDS)
            .readTimeout(30, TimeUnit.SECONDS)
            .build();

        retrofit = new Retrofit.Builder()
            .baseUrl(BASE_URL)
            .client(client)
            .addConverterFactory(GsonConverterFactory.create())
            .build();
    }
    return retrofit.create(ApiService.class);
}
}

```

Using Retrofit in Activity:

```

public class MainActivity extends AppCompatActivity {

    private ApiService apiService;
    private RecyclerView recyclerView;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}

```

```

        apiService = RetrofitClient.getApiService();
        recyclerView = findViewById(R.id.recyclerView);

        fetchUsers();
    }

    private void fetchUsers() {
        apiService.getUsers().enqueue(new Callback<List<User>>() {
            @Override
            public void onResponse(Call<List<User>> call, Response<List<User>>
response) {
                if (response.isSuccessful()) {
                    List<User> users = response.body();
                    // Update UI with users list
                    updateRecyclerView(users);
                }
            }

            @Override
            public void onFailure(Call<List<User>> call, Throwable t) {
                Toast.makeText(MainActivity.this,
                    "Error: " + t.getMessage(), Toast.LENGTH_SHORT).show();
            }
        });
    }
}

```

2. Volley Implementation

Add dependency:

```

gradle
dependencies {

```

```
implementation 'com.android.volley:volley:1.2.1'

}
```

Volley Usage:

java

```
public class VolleyHelper {

    private RequestQueue requestQueue;

    private Context context;

    public VolleyHelper(Context context) {

        this.context = context;

        this.requestQueue = Volley.newRequestQueue(context);

    }

    public void makeGetRequest(String url, final VolleyCallback callback) {

        JsonObjectRequest request = new JsonObjectRequest(Request.Method.GET,
url, null,

        new Response.Listener<JSONObject>() {

            @Override

            public void onResponse(JSONObject response) {

                callback.onSuccess(response);

            }

        },

        new Response.ErrorListener() {

            @Override

            public void onErrorResponse(VolleyError error) {

                callback.onError(error.getMessage());

            }

        }

    ));

    requestQueue.add(request);

}
```

```

    }

    public void makePostRequest(String url, JSONObject jsonBody, final
VolleyCallback callback) {

        JsonObjectRequest request = new JsonObjectRequest(Request.Method.POST,
url, jsonBody,

            new Response.Listener<JSONObject>() {

                @Override

                public void onResponse(JSONObject response) {

                    callback.onSuccess(response);

                }

            },

            new Response.ErrorListener() {

                @Override

                public void onErrorResponse(VolleyError error) {

                    callback.onError(error.getMessage());

                }

            });

        requestQueue.add(request);

    }

    public void loadImage(String url, ImageView imageView, int defaultImage,
int errorImage) {

        ImageRequest request = new ImageRequest(url,

            new Response.Listener<Bitmap>() {

                @Override

                public void onResponse(Bitmap bitmap) {

                    imageView.setImageBitmap(bitmap);

                }

            }, 0, 0, null,

```

```

        new Response.ErrorListener() {
            public void onErrorResponse(VolleyError error) {
                imageView.setImageResource(errorImage);
            }
        });

    requestQueue.add(request);
}

interface VolleyCallback {
    void onSuccess(JSONObject result);
    void onError(String error);
}
}

```

3. Picasso for Image Loading

Add dependency:

```
dependencies {
```

```

    implementation 'com.squareup.picasso:picasso:2.8'
}

```

Usage:

```
// Basic image loading
```

```

Picasso.get()
    .load("https://example.com/image.jpg")
    .into(imageView);

```

```
// With placeholder and error handling
```

```

Picasso.get()
    .load("https://example.com/image.jpg")
    .placeholder(R.drawable.placeholder)
    .error(R.drawable.error)

```

```
.resize(200, 200)

.centerCrop()

.into(imageView);

// With callback
Picasso.get()

    .load(url)

    .into(imageView, new Callback() {

        @Override

        public void onSuccess() {

            // Image loaded successfully

        }

        @Override

        public void onError(Exception e) {

            // Handle error

        }

    });
```

4. Glide for Image Loading

Add dependency:

```
dependencies {

    implementation 'com.github.bumptech.glide:glide:4.15.1'

    annotationProcessor 'com.github.bumptech.glide:compiler:4.15.1'

}
```

Usage:

// Basic loading

```
Glide.with(context)

    .load("https://example.com/image.jpg")

    .into(imageView);
```

```
// Advanced usage
Glide.with(context)
    .load(url)
    .placeholder(R.drawable.placeholder)
    .error(R.drawable.error)
    .diskCacheStrategy(DiskCacheStrategy.ALL)
    .centerCrop()
    .circleCrop()
    .timeout(10000)
    .into(imageView);

// Load GIF
Glide.with(context)
    .asGif()
    .load("https://example.com/animation.gif")
    .into(imageView);
```

6. Notifications

Types of Notifications in Android

Android Notifications

- └─ Status Bar Notification
- └─ Heads-up Notification
- └─ Expandable Notification
- └─ Action Button Notification
- └─ Progress Notification
- └─ Messaging Style Notification
- └─ Big Picture Style Notification

Notification Components Diagram

text

Status Bar Icon		
Large Icon	Title	
	Text Content	
Action 1	Action 2	

Creating Basic Notification

Add dependency (for Android 13+):

```
dependencies {
    implementation 'androidx.core:core:1.9.0'
}
```

Notification Channels (Android 8.0+):

java

```
public class NotificationHelper {
    private Context context;
    private NotificationManager notificationManager;

    public static final String CHANNEL_ID = "default_channel";
    public static final String CHANNEL_NAME = "Default Channel";
    public static final String CHANNEL_DESCRIPTION = "This is default channel";

    public NotificationHelper(Context context) {
        this.context = context;
        this.notificationManager =
            (NotificationManager)
context.getSystemService(Context.NOTIFICATION_SERVICE);

        createNotificationChannel();
    }
}
```



```

    }

    @RequiresApi(api = Build.VERSION_CODES.O)
    private void createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel channel = new NotificationChannel(
                CHANNEL_ID,
                CHANNEL_NAME,
                NotificationManager.IMPORTANCE_DEFAULT
            );
            channel.setDescription(CHANNEL_DESCRIPTION);
            channel.enableLights(true);
            channel.setLightColor(Color.BLUE);
            channel.enableVibration(true);
            channel.setVibrationPattern(new long[]{100, 200, 300, 400});

            notificationManager.createNotificationChannel(channel);
        }
    }

    public void showSimpleNotification(String title, String content) {
        NotificationCompat.Builder builder = new
        NotificationCompat.Builder(context, CHANNEL_ID)
            .setSmallIcon(R.drawable.ic_notification)
            .setContentTitle(title)
            .setContentText(content)
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true);

        notificationManager.notify(1, builder.build());
    }

```

```
}  
}
```

Advanced Notification Examples

1. Notification with Actions:

```
public void showActionNotification() {  
    // Create intents for actions  
    Intent acceptIntent = new Intent(context, AcceptActivity.class);  
    PendingIntent acceptPendingIntent = PendingIntent.getActivity(  
        context, 0, acceptIntent, PendingIntent.FLAG_UPDATE_CURRENT |  
        PendingIntent.FLAG_IMMUTABLE);  
  
    Intent declineIntent = new Intent(context, DeclineActivity.class);  
    PendingIntent declinePendingIntent = PendingIntent.getActivity(  
        context, 1, declineIntent, PendingIntent.FLAG_UPDATE_CURRENT |  
        PendingIntent.FLAG_IMMUTABLE);  
  
    NotificationCompat.Builder builder = new  
        NotificationCompat.Builder(context, CHANNEL_ID)  
        .setSmallIcon(R.drawable.ic_notification)  
        .setContentTitle("Friend Request")  
        .setContentText("John Doe wants to be your friend")  
        .setPriority(NotificationCompat.PRIORITY_HIGH)  
        .setAutoCancel(true)  
        .addAction(R.drawable.ic_accept, "Accept", acceptPendingIntent)  
        .addAction(R.drawable.ic_decline, "Decline", declinePendingIntent);  
  
    notificationManager.notify(2, builder.build());  
}
```

2. Expandable Notification:

```
public void showExpandableNotification() {
```

```

        NotificationCompat.Builder builder = new
NotificationCompat.Builder(context, CHANNEL_ID)

        .setSmallIcon(R.drawable.ic_notification)

        .setContentTitle("New Messages")

        .setContentText("You have 3 new messages")

        .setStyle(new NotificationCompat.BigTextStyle()
                .bigText("Message 1: Hello\nMessage 2: How are you?\nMessage 3:
Let's meet"))

        .setPriority(NotificationCompat.PRIORITY_DEFAULT);

notificationManager.notify(3, builder.build());
}

```

3. Progress Notification:

```
public void showProgressNotification() {
```

```
    final int NOTIFICATION_ID = 4;
```

```

        NotificationCompat.Builder builder = new
NotificationCompat.Builder(context, CHANNEL_ID)

        .setSmallIcon(R.drawable.ic_notification)

        .setContentTitle("Download in progress")

        .setContentText("Downloading file...")

        .setPriority(NotificationCompat.PRIORITY_LOW)

        .setOngoing(true)

        .setProgress(100, 0, false);

notificationManager.notify(NOTIFICATION_ID, builder.build());

// Update progress
new Thread(() -> {
    for (int i = 0; i <= 100; i += 10) {
        try {

```

```

        Thread.sleep(500);

        builder.setProgress(100, i, false);

        builder.setContentText("Downloaded " + i + "%");

        notificationManager.notify(NOTIFICATION_ID, builder.build());
    } catch (InterruptedException e) {
        e.printStackTrace();
    }
}

builder.setContentText("Download complete")
    .setProgress(0, 0, false)
    .setOngoing(false);

notificationManager.notify(NOTIFICATION_ID, builder.build());
}).start();
}

```

4. Messaging Style Notification:

```

public void showMessagingNotification() {

    NotificationCompat.MessagingStyle messagingStyle =
        new NotificationCompat.MessagingStyle("Me")
            .addMessage("Hello", System.currentTimeMillis(), "John")
            .addMessage("Hi there", System.currentTimeMillis(), "Me")
            .addMessage("How are you?", System.currentTimeMillis(), "John")
            .setConversationTitle("Chat with John");

    NotificationCompat.Builder builder = new
    NotificationCompat.Builder(context, CHANNEL_ID)
        .setSmallIcon(R.drawable.ic_notification)
        .setStyle(messagingStyle)
        .setPriority(NotificationCompat.PRIORITY_HIGH);

    notificationManager.notify(5, builder.build());
}

```

```
}
```

5. Big Picture Style:

```
public void showBigPictureNotification() {  
    Bitmap bigPicture = BitmapFactory.decodeResource(context.getResources(),  
        R.drawable.big_image);  
  
    NotificationCompat.Builder builder = new  
        NotificationCompat.Builder(context, CHANNEL_ID)  
        .setSmallIcon(R.drawable.ic_notification)  
        .setContentTitle("New Photo")  
        .setContentText("Check out this amazing photo")  
        .setStyle(new NotificationCompat.BigPictureStyle()  
            .bigPicture(bigPicture)  
            .bigLargeIcon(null))  
        .setPriority(NotificationCompat.PRIORITY_DEFAULT);  
  
    notificationManager.notify(6, builder.build());  
}
```

Requesting Notification Permission (Android 13+)

```
public class NotificationPermissionActivity extends AppCompatActivity {  
  
    private static final int PERMISSION_REQUEST_CODE = 100;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        checkNotificationPermission();  
    }  
}
```

```

private void checkNotificationPermission() {

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {

        if (ContextCompat.checkSelfPermission(this,

            Manifest.permission.POST_NOTIFICATIONS) !=
PackageManager.PERMISSION_GRANTED) {

            ActivityCompat.requestPermissions(this,

                new String[]{Manifest.permission.POST_NOTIFICATIONS},

                PERMISSION_REQUEST_CODE);

        } else {

            // Permission already granted

            showNotification();

        }

    } else {

        // Below Android 13, permission not required

        showNotification();

    }

}

@Override

public void onRequestPermissionsResult(int requestCode, String[]
permissions,

                                     int[] grantResults) {

    super.onRequestPermissionsResult(requestCode, permissions,
grantResults);

    if (requestCode == PERMISSION_REQUEST_CODE) {

        if (grantResults.length > 0 &&

            grantResults[0] == PackageManager.PERMISSION_GRANTED) {

            showNotification();

        }

    }

}

```

```

        } else {
            Toast.makeText(this, "Notification permission denied",
                Toast.LENGTH_SHORT).show();
        }
    }
}
}
}

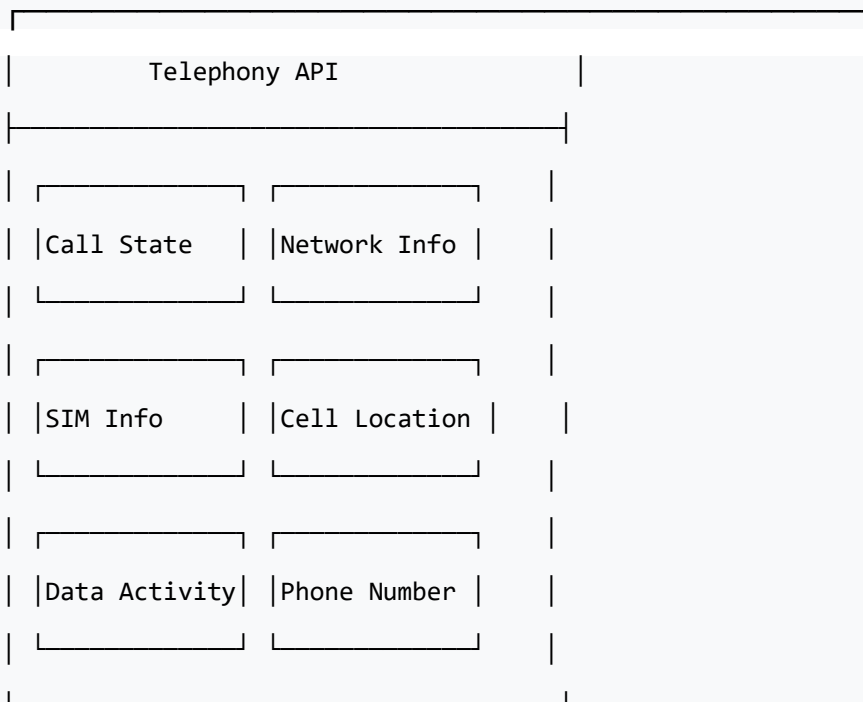
```

7. Telephony API

What is Telephony API?

Telephony API provides access to telephony features of the device including making calls, getting network information, and managing SIM cards.

Telephony Components Diagram



Required Permissions

```
<uses-permission android:name="android.permission.READ_PHONE_STATE" />
```

```
<uses-permission android:name="android.permission.CALL_PHONE" />
```

```
<uses-permission android:name="android.permission.READ_CALL_LOG" />
```

```
<uses-permission android:name="android.permission.READ_SMS" />
```

1. Making Phone Calls

```
public class PhoneCallHelper {

    public static void makeCall(Context context, String phoneNumber) {
        if (ContextCompat.checkSelfPermission(context,
            Manifest.permission.CALL_PHONE) ==
            PackageManager.PERMISSION_GRANTED) {

            Intent intent = new Intent(Intent.ACTION_CALL);
            intent.setData(Uri.parse("tel:" + phoneNumber));
            context.startActivity(intent);
        } else {
            // Request permission
            ActivityCompat.requestPermissions((Activity) context,
                new String[]{Manifest.permission.CALL_PHONE},
                REQUEST_CALL_PERMISSION);
        }
    }

    public static void dialNumber(Context context, String phoneNumber) {
        // Opens dialer with number pre-filled, but doesn't call automatically
        Intent intent = new Intent(Intent.ACTION_DIAL);
        intent.setData(Uri.parse("tel:" + phoneNumber));
        context.startActivity(intent);
    }
}
```

2. Getting Device Information

```
public class TelephonyInfo {
```



```
private TelephonyManager telephonyManager;

private Context context;

public TelephonyInfo(Context context) {
    this.context = context;
    this.telephonyManager = (TelephonyManager)
        context.getSystemService(Context.TELEPHONY_SERVICE);
}

@RequiresApi(api = Build.VERSION_CODES.Q)
public void getTelephonyInfo() {
    if (checkPermission()) {
        // Get device information
        String deviceId = telephonyManager.getDeviceId();
        String simSerialNumber = telephonyManager.getSimSerialNumber();
        String subscriberId = telephonyManager.getSubscriberId();
        String networkOperator = telephonyManager.getNetworkOperatorName();
        String simOperator = telephonyManager.getSimOperatorName();
        String networkCountryIso = telephonyManager.getNetworkCountryIso();

        // Get phone type
        int phoneType = telephonyManager.getPhoneType();
        String phoneTypeString = getPhoneTypeString(phoneType);

        // Get network type
        int networkType = telephonyManager.getNetworkType();
        String networkTypeString = getNetworkTypeString(networkType);

        // Get SIM state
        int simState = telephonyManager.getSimState();
    }
}
```

```

        String simStateString = getSimStateString(simState);

        // Get call state
        int callState = telephonyManager.getCallState();
        String callStateString = getCallStateString(callState);

        // Check roaming
        boolean isRoaming = telephonyManager.isNetworkRoaming();

        Log.d("TelephonyInfo", "Device ID: " + deviceId);
        Log.d("TelephonyInfo", "SIM Serial: " + simSerialNumber);
        Log.d("TelephonyInfo", "Subscriber ID: " + subscriberId);
        Log.d("TelephonyInfo", "Network Operator: " + networkOperator);
        Log.d("TelephonyInfo", "SIM Operator: " + simOperator);
        Log.d("TelephonyInfo", "Network Country: " + networkCountryIso);
        Log.d("TelephonyInfo", "Phone Type: " + phoneTypeString);
        Log.d("TelephonyInfo", "Network Type: " + networkTypeString);
        Log.d("TelephonyInfo", "SIM State: " + simStateString);
        Log.d("TelephonyInfo", "Call State: " + callStateString);
        Log.d("TelephonyInfo", "Is Roaming: " + isRoaming);
    }
}

private boolean checkPermission() {
    return ContextCompat.checkSelfPermission(context,
        Manifest.permission.READ_PHONE_STATE) ==
        PackageManager.PERMISSION_GRANTED;
}

private String getPhoneTypeString(int phoneType) {

```

```

        switch (phoneType) {
            case TelephonyManager.PHONE_TYPE_GSM: return "GSM";
            case TelephonyManager.PHONE_TYPE_CDMA: return "CDMA";
            case TelephonyManager.PHONE_TYPE_SIP: return "SIP";
            default: return "Unknown";
        }
    }

    private String getNetworkTypeString(int networkType) {
        switch (networkType) {
            case TelephonyManager.NETWORK_TYPE_GSM: return "GSM";
            case TelephonyManager.NETWORK_TYPE_LTE: return "LTE";
            case TelephonyManager.NETWORK_TYPE_UMTS: return "UMTS";
            case TelephonyManager.NETWORK_TYPE_CDMA: return "CDMA";
            case TelephonyManager.NETWORK_TYPE_EDGE: return "EDGE";
            case TelephonyManager.NETWORK_TYPE_GPRS: return "GPRS";
            case TelephonyManager.NETWORK_TYPE_5G: return "5G";
            default: return "Unknown";
        }
    }

    private String getSimStateString(int simState) {
        switch (simState) {
            case TelephonyManager.SIM_STATE_ABSENT: return "No SIM";
            case TelephonyManager.SIM_STATE_PIN_REQUIRED: return "PIN
Required";
            case TelephonyManager.SIM_STATE_PUK_REQUIRED: return "PUK
Required";
            case TelephonyManager.SIM_STATE_NETWORK_LOCKED: return "Network
Locked";
            case TelephonyManager.SIM_STATE_READY: return "Ready";

```

```

        default: return "Unknown";
    }
}

private String getCallStateString(int callState) {
    switch (callState) {
        case TelephonyManager.CALL_STATE_IDLE: return "Idle";
        case TelephonyManager.CALL_STATE_RINGING: return "Ringing";
        case TelephonyManager.CALL_STATE_OFFHOOK: return "Off Hook";
        default: return "Unknown";
    }
}
}

```

3. Call State Monitoring

```

public class CallStateListener extends PhoneStateListener {
    private Context context;

    public CallStateListener(Context context) {
        this.context = context;
    }

    @Override
    public void onCallStateChanged(int state, String phoneNumber) {
        super.onCallStateChanged(state, phoneNumber);

        switch (state) {
            case TelephonyManager.CALL_STATE_IDLE:
                Log.d("CallState", "Phone is idle");
                break;

```

```

        case TelephonyManager.CALL_STATE_RINGING:
            Log.d("CallState", "Incoming call from: " + phoneNumber);
            // Show notification for incoming call
            showIncomingCallNotification(phoneNumber);
            break;

        case TelephonyManager.CALL_STATE_OFFHOOK:
            Log.d("CallState", "Call is active");
            break;
    }
}

private void showIncomingCallNotification(String phoneNumber) {
    NotificationCompat.Builder builder = new
    NotificationCompat.Builder(context, "call_channel")
        .setSmallIcon(R.drawable.ic_call)
        .setContentTitle("Incoming Call")
        .setContentText("From: " + phoneNumber)
        .setPriority(NotificationCompat.PRIORITY_HIGH);

    NotificationManager notificationManager =
        (NotificationManager)
    context.getSystemService(Context.NOTIFICATION_SERVICE);
    notificationManager.notify(100, builder.build());
}
}

// Register the listener
TelephonyManager telephonyManager =
    (TelephonyManager) getSystemService(Context.TELEPHONY_SERVICE);
telephonyManager.listen(new CallStateListener(this),

```

```
PhoneStateListener.LISTEN_CALL_STATE);
```

4. Getting Cell Information

```
@RequiresApi(api = Build.VERSION_CODES.Q)
```

```
public class CellInfoHelper {
```

```
    public static void getCellInfo(Context context) {
```

```
        TelephonyManager telephonyManager =
```

```
            (TelephonyManager)
```

```
context.getSystemService(Context.TELEPHONY_SERVICE);
```

```
        if (checkPermission(context)) {
```

```
            List<CellInfo> cellInfoList = telephonyManager.getAllCellInfo();
```

```
            if (cellInfoList != null) {
```

```
                for (CellInfo cellInfo : cellInfoList) {
```

```
                    if (cellInfo instanceof CellInfoGsm) {
```

```
                        CellInfoGsm cellInfoGsm = (CellInfoGsm) cellInfo;
```

```
                        CellIdentityGsm cellIdentity =  
cellInfoGsm.getCellIdentity();
```

```
                        CellSignalStrengthGsm signalStrength =  
cellInfoGsm.getCellSignalStrength();
```

```
                            Log.d("CellInfo", "GSM Cell:");
```

```
                            Log.d("CellInfo", "CID: " + cellIdentity.getCid());
```

```
                            Log.d("CellInfo", "LAC: " + cellIdentity.getLac());
```

```
                            Log.d("CellInfo", "Signal Strength: " +  
signalStrength.getDbm() + " dBm");
```

```
                    } else if (cellInfo instanceof CellInfoLte) {
```

```
                        CellInfoLte cellInfoLte = (CellInfoLte) cellInfo;
```

```
                        CellIdentityLte cellIdentity =  
cellInfoLte.getCellIdentity();
```

```

        CellSignalStrengthLte signalStrength =
cellInfoLte.getCellSignalStrength();

        Log.d("CellInfo", "LTE Cell:");
        Log.d("CellInfo", "CI: " + cellIdentity.getCi());
        Log.d("CellInfo", "TAC: " + cellIdentity.getTac());
        Log.d("CellInfo", "Signal Strength: " +
signalStrength.getDbm() + " dBm");
        Log.d("CellInfo", "RSRP: " + signalStrength.getRsrp() +
" dBm");
        Log.d("CellInfo", "RSRQ: " + signalStrength.getRsrq() +
" dB");
    }
}
}
}

private static boolean checkPermission(Context context) {
    return ContextCompat.checkSelfPermission(context,
        Manifest.permission.ACCESS_FINE_LOCATION) ==
PackageManager.PERMISSION_GRANTED;
}
}

```

5. Sending SMS

```

public class SmsHelper {

    public static void sendSms(Context context, String phoneNumber, String
message) {
        if (ContextCompat.checkSelfPermission(context,
            Manifest.permission.SEND_SMS) == PackageManager.PERMISSION_GRANTED)
        {

```

```

SmsManager smsManager = SmsManager.getDefault();

// For long messages that need to be split
ArrayList<String> messageParts = smsManager.divideMessage(message);

if (messageParts.size() > 1) {
    // Send multipart SMS
    ArrayList<PendingIntent> sentIntents = new ArrayList<>();
    ArrayList<PendingIntent> deliveryIntents = new ArrayList<>();

    for (int i = 0; i < messageParts.size(); i++) {
        sentIntents.add(PendingIntent.getBroadcast(context, i,
            new Intent("SMS_SENT"), PendingIntent.FLAG_IMMUTABLE));
        deliveryIntents.add(PendingIntent.getBroadcast(context, i,
            new Intent("SMS_DELIVERED"),
PendingIntent.FLAG_IMMUTABLE));
    }

    smsManager.sendMultipartTextMessage(phoneNumber, null,
messageParts,
        sentIntents, deliveryIntents);
} else {
    // Send single SMS
    PendingIntent sentIntent = PendingIntent.getBroadcast(context,
0,
        new Intent("SMS_SENT"), PendingIntent.FLAG_IMMUTABLE);

    PendingIntent deliveryIntent =
PendingIntent.getBroadcast(context, 0,
        new Intent("SMS_DELIVERED"), PendingIntent.FLAG_IMMUTABLE);

    smsManager.sendTextMessage(phoneNumber, null, message,
        sentIntent, deliveryIntent);
}

```



```

    }

    // Register broadcast receivers for status
    registerSmsReceivers(context);

} else {

    // Request permission
    ActivityCompat.requestPermissions((Activity) context,
        new String[]{Manifest.permission.SEND_SMS},
        REQUEST_SMS_PERMISSION);
}

}

private static void registerSmsReceivers(Context context) {
    // Receiver for sent status
    context.registerReceiver(new BroadcastReceiver() {
        @Override
        public void onReceive(Context context, Intent intent) {
            switch (getResultCode()) {
                case Activity.RESULT_OK:
                    Toast.makeText(context, "SMS sent",
                        Toast.LENGTH_SHORT).show();
                    break;
                case SmsManager.RESULT_ERROR_GENERIC_FAILURE:
                    Toast.makeText(context, "Generic failure",
                        Toast.LENGTH_SHORT).show();
                    break;
                case SmsManager.RESULT_ERROR_NO_SERVICE:
                    Toast.makeText(context, "No service",
                        Toast.LENGTH_SHORT).show();
                    break;
            }
        }
    });
}

```

```
        }  
        }, new IntentFilter("SMS_SENT"));  
    }  
}
```

8. Google API

Introduction to Google APIs

Google provides various APIs for Android developers to integrate Google services into their applications.

Types of Google APIs

Google APIs

- |— Google Maps API
 - | |— Maps SDK
 - | |— Places API
 - | |— Directions API
- |— Google Sign-In
- |— Google Drive API
- |— Google Fit API
- |— Google Analytics
- |— Firebase Services
 - | |— Authentication
 - | |— Cloud Messaging
 - | |— Realtime Database
 - | |— Cloud Storage
- |— Google Play Services

Setting Up Google Play Services

1. Add dependency in build.gradle:

gradle

```
dependencies {  
  
    implementation 'com.google.android.gms:play-services-maps:18.1.0'
```

```
        implementation 'com.google.android.gms:play-services-location:21.0.1'
        implementation 'com.google.android.gms:play-services-auth:20.5.0'
        implementation 'com.google.android.gms:play-services-places:17.0.0'
    }
}
```

2. Add API key in AndroidManifest.xml:

xml

```
<meta-data
    android:name="com.google.android.geo.API_KEY"
    android:value="YOUR_API_KEY_HERE" />
```

1. Google Maps Integration

Activity Layout (activity_maps.xml):

```
<fragment xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:map="http://schemas.android.com/apk/res-auto"
    android:id="@+id/map"
    android:name="com.google.android.gms.maps.SupportMapFragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    map:cameraZoom="15"
    map:cameraTargetLat="37.7749"
    map:cameraTargetLng="-122.4194" />
```

Maps Activity:

public class MapsActivity extends AppCompatActivity

```
    implements OnMapReadyCallback, GoogleMap.OnMarkerClickListener {

    private GoogleMap mMap;
    private FusedLocationProviderClient fusedLocationClient;
    private LocationRequest locationRequest;
    private LocationCallback locationCallback;
```

```
@Override

protected void onCreate(Bundle savedInstanceState) {

    super.onCreate(savedInstanceState);

    setContentView(R.layout.activity_maps);


    // Initialize location client

    fusedLocationClient =
LocationServices.getFusedLocationProviderClient(this);


    // Get map fragment

    SupportMapFragment mapFragment = (SupportMapFragment)
        getSupportFragmentManager().findFragmentById(R.id.map);
    mapFragment.getMapAsync(this);


    createLocationRequest();
}


@Override

public void onMapReady(GoogleMap googleMap) {

    mMap = googleMap;


    // Enable zoom controls

    mMap.getUiSettings().setZoomControlsEnabled(true);
    mMap.getUiSettings().setCompassEnabled(true);
    mMap.getUiSettings().setMyLocationButtonEnabled(true);


    // Set map type

    mMap.setMapType(GoogleMap.MAP_TYPE_NORMAL);


    // Check location permission
```

```

if (checkLocationPermission()) {
    mMap.setMyLocationEnabled(true);
    getCurrentLocation();
}

// Set marker click listener
mMap.setOnMarkerClickListener(this);

// Add a marker
addMarker(new LatLng(37.7749, -122.4194), "San Francisco", "Beautiful
city");
}

private void addMarker(LatLng position, String title, String snippet) {
    MarkerOptions markerOptions = new MarkerOptions()
        .position(position)
        .title(title)
        .snippet(snippet)
        .icon(BitmapDescriptorFactory.defaultMarker(BitmapDescriptorFactory.HUE_RED));

    mMap.addMarker(markerOptions);
}

private void getCurrentLocation() {
    try {
        fusedLocationClient.getLastLocation()
            .addOnSuccessListener(this, location -> {
                if (location != null) {
                    LatLng currentLocation = new
                    LatLng(location.getLatitude(),

```

```

        location.getLongitude());
        mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(
            currentLocation, 15));

        addMarker(currentLocation, "You are here", "Current
location");
    } else {
        requestLocationUpdates();
    }
});
} catch (SecurityException e) {
    e.printStackTrace();
}
}

private void createLocationRequest() {
    locationRequest = LocationRequest.create();
    locationRequest.setInterval(10000);
    locationRequest.setFastestInterval(5000);
    locationRequest.setPriority(LocationRequest.PRIORITY_HIGH_ACCURACY);
}

private void requestLocationUpdates() {
    try {
        locationCallback = new LocationCallback() {
            @Override
            public void onLocationResult(LocationResult locationResult) {
                if (locationResult == null) {
                    return;
                }
            }
        };
    }
}

```

```

        for (Location location : locationResult.getLocations()) {
            LatLng currentLocation = new
LatLng(location.getLatitude(),
            location.getLongitude());
            mMap.animateCamera(CameraUpdateFactory.newLatLngZoom(
                currentLocation, 15));
        }
    }
};

fusedLocationClient.requestLocationUpdates(locationRequest,
    locationCallback, Looper.getMainLooper());

} catch (SecurityException e) {
    e.printStackTrace();
}
}

private boolean checkLocationPermission() {
    if (ContextCompat.checkSelfPermission(this,
        Manifest.permission.ACCESS_FINE_LOCATION) !=
PackageManager.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(this,
            new String[]{Manifest.permission.ACCESS_FINE_LOCATION},
            LOCATION_PERMISSION_REQUEST_CODE);
        return false;
    }
    return true;
}

```

```

@Override

public boolean onMarkerClick(Marker marker) {

    // Handle marker click

    Toast.makeText(this, "Clicked: " + marker.getTitle(),
Toast.LENGTH_SHORT).show();

    return false;

}

}

```

2. Google Sign-In Integration

Add dependency:

```
dependencies {
```

```

    implementation 'com.google.android.gms:play-services-auth:20.5.0'

}

```

Google Sign-In Activity:

```
public class GoogleSignInActivity extends AppCompatActivity {
```

```

    private static final int RC_SIGN_IN = 9001;

    private GoogleSignInClient mGoogleSignInClient;

    private TextView nameTextView;

    private TextView emailTextView;

    private ImageView profileImageView;

    @Override

    protected void onCreate(Bundle savedInstanceState) {

        super.onCreate(savedInstanceState);

        setContentView(R.layout.activity_google_signin);

        nameTextView = findViewById(R.id.name);

        emailTextView = findViewById(R.id.email);

        profileImageView = findViewById(R.id.profile_image);

```



```

        // Configure Google Sign-In

        GoogleSignInOptions gso = new
        GoogleSignInOptions.Builder(GoogleSignInOptions.DEFAULT_SIGN_IN)

            .requestEmail()

            .requestProfile()

            .requestId()

            .build();

        mGoogleSignInClient = GoogleSignIn.getClient(this, gso);

        findViewById(R.id.sign_in_button).setOnClickListener(v -> signIn());
        findViewById(R.id.sign_out_button).setOnClickListener(v -> signOut());
    }

    @Override
    protected void onStart() {
        super.onStart();

        // Check if user is already signed in

        GoogleSignInAccount account =
        GoogleSignIn.getLastSignedInAccount(this);

        updateUI(account);
    }

    private void signIn() {
        Intent signInIntent = mGoogleSignInClient.getSignInIntent();
        startActivityForResult(signInIntent, RC_SIGN_IN);
    }

    private void signOut() {
        mGoogleSignInClient.signOut()

            .addOnCompleteListener(this, task -> {

```

```

        updateUI(null);

        Toast.makeText(this, "Signed out successfully",
Toast.LENGTH_SHORT).show();

    });

}

@Override

protected void onActivityResult(int requestCode, int resultCode, Intent
data) {

    super.onActivityResult(requestCode, resultCode, data);

    if (requestCode == RC_SIGN_IN) {

        Task<GoogleSignInAccount> task =
GoogleSignIn.getSignedInAccountFromIntent(data);

        handleSignInResult(task);

    }

}

private void handleSignInResult(Task<GoogleSignInAccount> completedTask) {

    try {

        GoogleSignInAccount account =
completedTask.getResult(ApiException.class);

        updateUI(account);

    } catch (ApiException e) {

        Log.w("GoogleSignIn", "signInResult:failed code=" +
e.getStatusCode());

        updateUI(null);

    }

}

private void updateUI(GoogleSignInAccount account) {

    if (account != null) {

```

```
// User is signed in
String personName = account.getDisplayName();
String personEmail = account.getEmail();
String personId = account.getId();
Uri personPhoto = account.getPhotoUrl();

nameTextView.setText("Name: " + personName);
emailTextView.setText("Email: " + personEmail);

// Load profile image
if (personPhoto != null) {
    Glide.with(this)
        .load(personPhoto)
        .circleCrop()
        .into(profileImageView);
}

findViewById(R.id.sign_in_button).setVisibility(View.GONE);
findViewById(R.id.sign_out_button).setVisibility(View.VISIBLE);

} else {
    // User is signed out
    nameTextView.setText("Not signed in");
    emailTextView.setText("");
    profileImageView.setImageResource(R.drawable.ic_default_profile);

    findViewById(R.id.sign_in_button).setVisibility(View.VISIBLE);
    findViewById(R.id.sign_out_button).setVisibility(View.GONE);
}
}
```

```
}
```

3. Google Places API

```
public class PlacesActivity extends AppCompatActivity {
```

```
    private static final int AUTOCOMPLETE_REQUEST_CODE = 1;
    private EditText locationEditText;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_places);

        locationEditText = findViewById(R.id.location_input);

        // Initialize Places
        Places.initialize(getApplicationContext(), "YOUR_API_KEY");

        // Set up autocomplete
        locationEditText.setFocusable(false);
        locationEditText.setOnClickListener(v -> {
            List<Place.Field> fields = Arrays.asList(
                Place.Field.ID,
                Place.Field.NAME,
                Place.Field.ADDRESS,
                Place.Field.LAT_LNG
            );

            Intent intent = new Autocomplete.IntentBuilder(
                AutocompleteActivityMode.FULLSCREEN, fields)
                .build(this);
```

```

        startActivityForResult(intent, AUTOCOMPLETE_REQUEST_CODE);
    });
}

@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    if (requestCode == AUTOCOMPLETE_REQUEST_CODE) {
        if (resultCode == RESULT_OK) {
            Place place = Autocomplete.getPlaceFromIntent(data);

            locationEditText.setText(place.getName() + ", " +
            place.getAddress());

            // Get location details
            LatLng latLng = place.getLatLng();
            Log.d("Places", "Place: " + place.getName() + ", " +
            place.getId());
            Log.d("Places", "Lat: " + latLng.latitude + ", Lng: " +
            latLng.longitude);

        } else if (resultCode == AutocompleteActivity.RESULT_ERROR) {
            Status status = Autocomplete.getStatusFromIntent(data);
            Toast.makeText(this, "Error: " + status.getStatusMessage(),
                Toast.LENGTH_SHORT).show();
        }
    }
    super.onActivityResult(requestCode, resultCode, data);
}
}

```

4. Google Directions API

```

public class DirectionsHelper {

```

```

    private static final String BASE_URL =
"https://maps.googleapis.com/maps/api/directions/";

    public interface DirectionsCallback {

        void onSuccess(String polyline, List<LatLng> route);

        void onError(String error);

    }

    public static void getDirections(String origin, String destination,
                                    DirectionsCallback callback) {

        String url = BASE_URL + "json?" +
            "origin=" + origin +
            "&destination=" + destination +
            "&mode=driving" +
            "&key=YOUR_API_KEY";

        RequestQueue queue =
Volley.newRequestQueue(MyApplication.getContext());

        JsonObjectRequest request = new JsonObjectRequest(Request.Method.GET,
url, null,
            response -> {
                try {
                    JSONArray routes = response.getJSONArray("routes");

                    if (routes.length() > 0) {
                        JSONObject route = routes.getJSONObject(0);

                        JSONObject overviewPolyline =
route.getJSONObject("overview_polyline");

                        String polyline = overviewPolyline.getString("points");

                        // Decode polyline

                        List<LatLng> path = decodePoly(polyline);

```

```

        callback.onSuccess(polyline, path);
    }
} catch (JSONException e) {
    callback.onError(e.getMessage());
}
},
error -> callback.onError(error.getMessage()));

queue.add(request);
}

private static List<LatLng> decodePoly(String encoded) {
    List<LatLng> poly = new ArrayList<>();
    int index = 0, len = encoded.length();
    int lat = 0, lng = 0;

    while (index < len) {
        int b, shift = 0, result = 0;
        do {
            b = encoded.charAt(index++) - 63;
            result |= (b & 0x1f) << shift;
            shift += 5;
        } while (b >= 0x20);
        int dlat = ((result & 1) != 0 ? ~(result >> 1) : (result >> 1));
        lat += dlat;

        shift = 0;
        result = 0;
        do {

```

```

        b = encoded.charAt(index++) - 63;
        result |= (b & 0x1f) << shift;
        shift += 5;
    } while (b >= 0x20);
    int dlng = ((result & 1) != 0 ? ~(result >> 1) : (result >> 1));
    lng += dlng;

    LatLng p = new LatLng((((double) lat / 1E5)), (((double) lng /
1E5)));
    poly.add(p);
}
return poly;
}

// Draw route on map
public static void drawRoute(GoogleMap map, List<LatLng> points) {
    PolylineOptions polylineOptions = new PolylineOptions()
        .addAll(points)
        .width(10)
        .color(Color.BLUE)
        .geodesic(true);

    map.addPolyline(polylineOptions);
}
}

```

5. Google Fit API

```

@RequiresApi(api = Build.VERSION_CODES.Q)
public class FitnessHelper {

    private GoogleSignInAccount account;

```



```

    public void connectToFitness(Activity activity) {
        GoogleSignInOptions gso = new
GoogleSignInOptions.Builder(GoogleSignInOptions.DEFAULT_SIGN_IN)
            .requestScopes(new Scope(Scopes.FITNESS_ACTIVITY_READ))
            .requestScopes(new Scope(Scopes.FITNESS_ACTIVITY_WRITE))
            .requestScopes(new Scope(Scopes.FITNESS_BODY_READ))
            .requestScopes(new Scope(Scopes.FITNESS_LOCATION_READ))
            .build();

        GoogleSignIn.getClient(activity, gso)
            .silentSignIn()
            .addOnSuccessListener(account -> {
                this.account = account;
                readStepData();
            })
            .addOnFailureListener(e -> {
                // User needs to sign in
                activity.startActivityForResult(
                    GoogleSignIn.getClient(activity, gso).getSignInIntent(),
                    FITNESS_SIGN_IN_REQUEST);
            });
    }

    public void readStepData() {
        // Get today's date at midnight
        Calendar cal = Calendar.getInstance();
        Date now = new Date();
        cal.setTime(now);
        long endTime = cal.getTimeInMillis();
    }

```

```

        cal.set(Calendar.HOUR_OF_DAY, 0);
        cal.set(Calendar.MINUTE, 0);
        cal.set(Calendar.SECOND, 0);
        long startTime = cal.getTimeInMillis();

        DataReadRequest readRequest = new DataReadRequest.Builder()
            .read(DataType.TYPE_STEP_COUNT_DELTA)
            .setTimeRange(startTime, endTime, TimeUnit.MILLISECONDS)
            .build();

        Fitness.getHistoryClient(MyApplication.getContext(), account)
            .readData(readRequest)
            .addOnSuccessListener(response -> {
                int totalSteps = 0;

                for (DataSet dataSet : response.getDataSets()) {
                    for (DataPoint dp : dataSet.getDataPoints()) {
                        for (Field field : dp.getDataType().getFields()) {
                            int steps = dp.getValue(field).asInt();
                            totalSteps += steps;
                        }
                    }
                }

                Log.d("Fitness", "Total steps today: " + totalSteps);
            })
            .addOnFailureListener(e ->
                Log.e("Fitness", "Failed to read steps", e));
    }
}

```

6. Google Cloud Messaging (Firebase Cloud Messaging)

Add dependency:

gradle

```
dependencies {  
    implementation 'com.google.firebase:firebase-messaging:23.1.2'  
}
```

FCM Service:

public class MyFirebaseMessagingService extends FirebaseMessagingService {

```
    @Override  
    public void onMessageReceived(RemoteMessage remoteMessage) {  
        // Handle FCM messages  
        if (remoteMessage.getNotification() != null) {  
            String title = remoteMessage.getNotification().getTitle();  
            String body = remoteMessage.getNotification().getBody();  
            showNotification(title, body);  
        }  
  
        // Handle data messages  
        if (remoteMessage.getData().size() > 0) {  
            Map<String, String> data = remoteMessage.getData();  
            // Process data  
        }  
    }  
  
    @Override  
    public void onNewToken(String token) {  
        // Send token to your server  
        sendRegistrationToServer(token);  
    }  
}
```

```
private void showNotification(String title, String body) {  
    NotificationCompat.Builder builder = new  
NotificationCompat.Builder(this, "fcm_channel")  
        .setSmallIcon(R.drawable.ic_notification)  
        .setContentTitle(title)  
        .setContentText(body)  
        .setPriority(NotificationCompat.PRIORITY_HIGH)  
        .setAutoCancel(true);  
  
    NotificationManager manager =  
        (NotificationManager)  
getSystemService(Context.NOTIFICATION_SERVICE);  
    manager.notify(0, builder.build());  
}  
}
```